

PGM Image Processing Library

UML Class Diagram

«struct»

Image

- rows : int
- cols : int
- max_gray : int
- pixels : int**

«Part 1: Initialization»

+ create_image(cols : int, rows : int, max_gray : int) : Image*

«Part 2: Copying and Information Retrieval»

+ copy_image(source : Image*) : Image*

+ get_rows(img : Image*) : int

+ get_cols(img : Image*) : int

+ get_max_gray(img : Image*) : int

«Part 3: Rotation and Negation»

+ negate_image(img : Image*) : Image*

+ rotate_image(img : Image*, angle : float) : void

«Part 4: Pixel Operations»

+ is_valid(img : Image*, r : int, c : int) : int

+ get_pixel(img : Image*, r : int, c : int) : int

+ set_pixel(img : Image*, r : int, c : int, val : int) : void

+ print_image(img : Image*, title : const char*) : void

«Part 5: Encoding/Decoding - Planned»

+ encode_message(img : Image*, msg : const char*) : void

+ decode_message(img : Image*) : char*

Dependencies:

- stdio.h (printf, FILE*)
- stdlib.h (malloc, free, calloc)
- math.h (cos, sin)

Memory Model:

pixels → int* [] → int []
(2D dynamic array)

Function Reference

Function	Return	Description
create_image	Image*	Allocates Image struct and 2D pixel array, initializes all pixels to 0
copy_image	Image*	Creates deep copy with independent pixel memory
get_rows	int	Returns img→rows
get_cols	int	Returns img→cols
get_max_gray	int	Returns img→max_gray
negate_image	Image*	Creates new image where pixel = max_gray - original_pixel
rotate_image	void	Rotates in-place using cos/sin transformation around center
is_valid	int	Returns 1 if (r,c) within bounds, 0 otherwise
get_pixel	int	Returns pixel value at (r,c) or 0 if out of bounds
set_pixel	void	Sets pixel at (r,c) to val if within bounds
print_image	void	Outputs image dimensions and pixel grid to console

Part 5: Steganography Strategy

Encoding: Split 8-bit ASCII into 4×2-bit chunks, embed in LSB of 4 consecutive pixels

Decoding: Extract LSB (pixel & 3) from 4 pixels, reconstruct: $(b1 \ll 6) \mid (b2 \ll 4) \mid (b3 \ll 2) \mid b4$

Terminator: Null character '\0' marks end of message

Capacity: $(\text{rows} \times \text{cols}) / 4 - 1$ characters maximum